

Assignment 1: Text Normalization, Tokenization, and Corpus Analysis

Computational Linguistics – I

Deadline

February 02, 2026 (Monday), 11:59 PM

Objective

This assignment introduces the early stages of computational language processing: **text normalization**, **tokenization**, and basic **corpus analysis**. The focus is on understanding how linguistic decisions are translated into computational rules, rather than on writing large amounts of code.

Important Note

- This assignment follows a **scaffolded programming approach**.
- Approximately **70–80% of the code is provided**.
- You are expected to:
 - understand the given code,
 - complete marked TODO sections,
 - justify linguistic choices in writing.
- Teaching Assistants will conduct tutorial sessions to explain the provided code.

Dataset Allocation

You must process **only one** file from the dataset. The dataset consists of 18 unique corpus files derived from the Brown Corpus. Use the data assigned to you according to your roll number.

- **Download the Dataset:** Click here to access the Shared Folder

Task 1: Text Normalization (25 Marks)

You are given a partially implemented normalization pipeline. Complete the functions marked as TODO.

Provided Code Skeleton

```
import re

def normalize_whitespace(text):
    """
    Removes extra whitespace.
    """
    text = re.sub(r"\s+", " ", text)
    return text.strip()

def normalize_case(text):
    """
    Converts text to lowercase.
    """
    return text.lower()

def normalize_contractions(text):
    """
    TODO:
    Expand common English contractions.
    Example:
    isn't -> is not
    don't -> do not
    """
    # TODO: write regex-based rules
    return text

def normalize_quotes(text):
    """
    TODO:
    Convert different quotation marks to standard double quotes.
    """
    # TODO: handle curly quotes, single quotes, etc.
    return text

def normalize_text(text):
    """
    Complete normalization pipeline.
    """
    text = normalize_whitespace(text)
    text = normalize_case(text)
    text = normalize_contractions(text)
    text = normalize_quotes(text)
    return text
```

What You Need to Do

- Complete the TODO sections
- Run the normalisation code on the dataset provided.
- Explain your linguistic reasoning in the README

Task 2: Tokenization (25 Marks)

You are provided with a basic tokenizer structure. Refine it to handle linguistic edge cases. Then, run the tokenizer on the given dataset.

Provided Code Skeleton

```
import re

def sentence_tokenize(text):
    """
    Splits text into sentences.
    """
    parts = re.split(r'([.!?])', text)
    sentences = []
    # TODO: merge punctuation with sentences correctly
    return sentences

def word_tokenize(sentence):
    """
    Splits sentence into word tokens.
    """
    # TODO:
    # - preserve contractions
    # - handle hyphenated words
    # - separate punctuation
    tokens = re.findall(r"\w+|[\^\w\s]", sentence)
    return tokens
```

Expected Behavior

Input: Mr. Smith's self-driving car isn't working!

Output tokens:

[Mr., Smith's, self-driving, car, is, not, working, !]

Task 3: Linguistic Reflection (20 Marks)

Answer the following questions (1–2 paragraphs each):

1. Why is tokenization not a purely technical problem?
2. Give one example from an Indian language where whitespace tokenization fails.
3. How would you modify the given tokenizer for that language?

Task 4: Corpus Analysis and Zipf's Law (20 Marks)

In this task, you will explore frequency distributions in natural language and observe **Zipf's Law** empirically.

Provided Code Skeleton

```
from collections import Counter
import math

def compute_word_frequencies(tokens):
    return Counter(tokens)

def compute_rank_frequency(freq_dict):
    sorted_items = sorted(freq_dict.items(),
                           key=lambda x: x[1],
                           reverse=True)

    rank_freq = []
    for rank, (word, freq) in enumerate(sorted_items, start=1):
        rank_freq.append((rank, freq))
    return rank_freq

def log_transform(rank_freq):
    """
    TODO:
    Convert rank and frequency into log-log values.
    """
    log_rank_freq = []
    for rank, freq in rank_freq:
        # TODO: apply log transformation
        pass
    return log_rank_freq
```

What You Need to Do

- Use tokens produced in Task 2
- Complete the `log_transform` function
- Identify the top 20 most frequent tokens

Analysis Questions

Answer briefly in the README:

1. Which types of words dominate high-frequency ranks?
2. What kinds of words appear in the long tail?
3. How does normalization affect the frequency distribution?
4. Would Zipf's Law differ for morphologically rich Indian languages? Explain.

Note: Plotting is optional.

Task 5: Testing and Error Analysis (10 Marks)

- Test your pipeline on at least **10 sentences with minimum of 7 tokens** from your assigned dataset
- Identify:
 - cases where rules work well

- cases where they fail
- Briefly explain why these failures occur

Submission Instructions

Submit a single ZIP file named:

`<rollnumber>_assignment1.zip`

The ZIP file must contain:

- `normalizer.py`
- `tokenizer.py`
- `corpus-analysis.py`
- `README.pdf` / `README.md` (mandatory)

Evaluation Scheme

Component	Marks
Text Normalization	25
Tokenization	25
Linguistic Reflection	20
Zipf & Corpus Analysis	20
Testing & Error Analysis	10
Total	100

Academic Integrity

This assignment must be entirely your own work. Understanding and extending the provided code is expected; copying solutions is not permitted.

Suggested Reading

- Jurafsky & Martin, *Speech and Language Processing*, Chapters 2–3
- Python Regular Expressions Documentation: <https://docs.python.org/3/library/re.html>